

PROGETTO DI INFORMATICA

Componenti del gruppo:

1- Raffaele Piovesan , Matricola : 910513

2- Andreotta Lorenzo, Matricola: 912983

3- Mazzitelli Riccardo, Matricola: 912166

Architettura strutture dati (STRUCT):

STRUCT PLAYER:

Abbiamo implementato per la gestione del protagonista una struttura (struct) "Player".

In questa struct troviamo:

- **Stats** -> come gli HP (punti vita) massimi , quelli correnti e le monete.
- **Delle Flag** -> per l'inventario e per capire se la struct player è stata istanziata o meno.

STRUCT DUNGEON:

Abbiamo una gestione da due livelli delle stanze:

- **stanza_base** -> Un array statico che funge come "database" per i modelli predefiniti
- **stanzalivello (stanza)** -> Invece quest'ultima è l'implementazione effettiva in game , ha dei parametri in più come "visitata" e "obiettivo_missione" per tracciare i progressi del giocatore in game.

Creazione dei Dungeon e logica di generazione:

La funzione "generaStanze" popola un array di "Max_stanze" basandosi su quale missione il giocatore ha scelto.

MECCANICA DI GENERAZIONE:

Il dungeon non è generato casualmente ma ha un vincolo di causalità.

- **Lancio del dado** -> Tramite la funzione "rollareDadi(true)" , il sistema estrae un indice per pescare dai template delle stanze.
- **Forzatura degli obiettivi** -> Per evitare che un dungeon sia imbattibile o privo dei boss principali , il codice include dei controlli:
 - **Nella palude** -> Se non ci sono abbastanza "generali_orco" o se ci sono poche stanze , il sistema smette di inserire con i dadi ma usa delle forzature.
 - **Nella grotta** -> L'ultima stanza è impostata manualmente come "Drago Antico" per garantire il che il giocatore arrivi ad avere la possibilità di ottenere "Spada dell'eroe".

Sistema dei Menu' e Game Loop:

Ogni menù (Iniziale , Partita , Viaggio ect..) restituisce un "int".

Seguito da dei "switch-case" che decidono quale funzione chiamare , cercando di mantenere la logica pulita.

Poi abbiamo anche ulteriori controlli sul "Player" come nel "Menu Viaggi" che controlla se il "Player.has_key" è riuscito ad avere la chiave del Castello e quindi di proseguire e finire il "game_loop".

Sistema del negozio:

Il negozio funge da "Hub" commerciale tra una missione e l'altra. La logica è progettata per essere dinamica e sicura:

- **Verifica disponibilità** -> il menu mostra determinati oggetti come Spada, Armatura solo se il giocatore non li possiede già ("!Player.has_sword").
- **Controllo Transizione** -> Prima di ogni acquisto il sistema verificherà che il "Player" abbia abbastanza monete per permettersi di comprare ciò che vuole, nel caso abbia le monete giuste, le sottrae e aggiorna lo stato del "Player".
- **Interfaccia Dinamica** -> "max_neg" è una variabile che limita gli input dell'utente solo alle opzioni effettivamente visibili a schermo.

Sistema della cura:

Invece di essere una cura fissa, il gioco utilizza la funzione "rollareDadi(true)" per simulare un lancio del dado da 6 facce.

- **Effetto** -> Lancia il dado a 6 facce, il risultato che esce viene aggiunto ai "current_hp" del giocatore.
- **Consumo** -> Viene incrementata la variabile "num_pozioni" solo dopo l'effetto.

Bozza dell'indovinello di Padova:

Per la sfida contro il "Drago antico", si ha implementato come da consegna l'algoritmo della successione di Padova, che a differenza di Fibonacci questa sequenza cresce lentamente ed è definita dalla seguente formula:

$$P(n) = P(n-2) + P(n+3)$$

con dei valori iniziali: $P(0) = 1$, $P(1) = 1$, $P(2) = 1$.

LOGICA DI VERIFICA:

- **la funzione padovan** -> è stata calcolata in modo ricorsivo.
- **is_in_padovan(num, n)** -> controlla se il numero inserito dal giocatore è un termine valido della serie, permettendo al drago di convalidare la risposta dell'indovinello.

Sistema dell'esplorazione dungeon e Combattimento:

Una volta scelto il dungeon da esplorare e generate le stanze viene chiamata la funzione esplora_stanze che prende in input il numero del livello (1 per palude, 2 per magione, 3 per grotta) e la funzione ha un while che richiama il menù che si occupa delle decisioni all'interno delle esplorazioni (menuMissione) che ritorna true se si ha scelto di affrontare una stanza o false se il giocatore ha fatto un'altra azione questo per far avanzare il contatore che gestisce le stanze. Finito di esplorare le stanze ci sono dei controlli per verificare che il player abbia completato gli obiettivi del dungeon oppure abbia abbandonato il dungeon prima di aver completato gli obiettivi.

Il menuMissione permette al giocatore di fare varie cose in particolare:

- curarsi: permette di usare una pozione di cura senza passare per l'inventario
- usare il negozio: per acquistare armi armature e pozioni di cura
- abbandonare missione: spendendo monete è possibile abbandonare il dungeon
- esplorare una stanza: esplora una stanza generata

gestisci combattimento: riceve in input la stanza da esplorare

in base al tipo avvia una delle boss fight un combattimento tradizionale oppure, nel caso di una trappola, danneggia il giocatore

la funzione richiama funzioni helper per danneggiare il giocatore o assegnare delle ricompense

Drago caverna di cristalli:

usa la sequenza di Padovan, il giocatore dice un numero se appartiene alla sequenza vince altrimenti subisce danni

Signore Oscuro:

Sasso carta forbici e in 5 round il primo che raggiunge 3 vittorie vince se ciò non avviene c'è un twist particolare.

Altre funzioni:

Aspetta invio per permettere al giocatore di leggere i messaggi stampati dal gioco.

Sistema di salvataggio

Libreria vector.h:

Necessaria all'interazione con i file di salvataggio. Ogni volta che si avvia il gioco, l'array data verrà caricato con delle struct presenti in un file indice, in modo da poter mostrare le statistiche di ciascun salvataggio. I file di salvataggio avranno un nome del tipo "salvataggio0.bin", quindi l'indice di data verrà usato per la stringa da inserire in fopen().

Si tratta di una rivisitazione dell'array dinamico (sostitutivo del vector di C++ dato che in C non è presente) visto a lezione nel modulo di teoria di IaP.

È presente una struct chiamata fileFormat, il cui scopo è quello memorizzare i dati principali del salvataggio (data, pVita, monete, oggetti...), che verranno mostrati quando dal menù verrà selezionata l'opzione per caricare/eliminare un salvataggio.

Funzioni:

- **saveToFile:** sono presenti due puntatori di tipo FILE, uno per la scrittura effettiva dei dati del giocatore nel salvataggio ed uno per scrivere i dati del salvataggio nel file index. Per costruire il nome effettivo del salvataggio (salvataggio<n>.bin) è presente un array di char chiamato fileName, formattato da sprintf per inserire il numero di salvataggio (preso dal numero di elementi presenti nel vector). Una volta generato il nome del file viene utilizzata fopen(); nella modalità write binary, in modo da creare (dato che ancora non esiste) e scrivere, tramite fwrite che va ad inserire un record contenente una struct di tipo player, il file di salvataggio. Terminato questo processo (e quindi dopo aver chiuso lo stream di fp, dato che non è possibile avere due stream aperti contemporaneamente) si procede a scrivere i dati del salvataggio nel file indice (tramite append binary, dato che write binary sovrascrive l'intero contenuto del file) e ad aggiungerli all'interno di vector. Ciò avviene alla fine perché bisogna assicurarsi che le scritture su file abbiano avuto successo.

- **loadFromFile:** è presente un controllo iniziale per verificare che il salvataggio richiesto dal player sia valido, dopodiché come nella funzione saveToFile viene costruito all'interno di una stringa il nome (salvataggio<n>.bin) del file di salvataggio a cui accedere. Tramite la fread() si ottengono i dati della struct player (da sostituire a quelli della struttura player attuale) per poi chiudere lo stream.
- **loadSavesVector:** si occupa di leggere i dati dei salvataggi presenti all'interno del file indice e di caricare le varie struct (ad eccezione di quelle flaggate come eliminate) all'interno del vector utilizzando la funzione vAdd(v, data) della libreria vector.h. L'apertura dello stream è stata effettuata tramite l'opzione "read binary" in quanto il file non deve essere modificato.
- **renameFiles:** necessaria per l'eliminazione di un file di salvataggio. Quando un file viene eliminato, viene rimosso anche dal vector e se non venisse aggiornato il valore presente alla fine del nome del salvataggio si avrebbe una incoerenza tra i file presenti nella directory e l'indice delle strutture salvate nel vector. Nella stringa fileRename viene messo il nome del file da modificare, mentre nella stringa newFileName il nuovo nome del file (<n> decrementato di 1), per poi rinominare effettivamente il file con la funzione rename.
- **deleteFromIndex:** elimina dal file indice le informazioni del salvataggio da eliminare. Lo stream viene aperto in "r+" per poter apportare delle modifiche ai record, nello specifico il campo "deleted" della struct per flaggare che quel record non è più presente. La ricerca all'interno del file viene effettuata comparando la data poiché univoca. Dato che fread(); sposta automaticamente il puntatore ad ogni lettura, è necessario utilizzare fseek(); per tornare indietro di un record e modificare il flag "deleted" (la fwrite sposterà poi di nuovo il puntatore in avanti di un record),
- **deleteSave:** viene usato un flag isDeletedFromIndex il cui valore verrà assegnato dal ritorno della funzione deleteFromIndex per assicurarsi che l'eliminazione del file indice abbia avuto successo. Si usa poi la stringa fileDelete e la funzione sprintf(); per costruire il nome del salvataggio da eliminare (nSave viene decrementato di uno poiché è stato scelto un sistema zero-indexed per i nomi dei file di salvataggio, in modo da essere coerenti con gli indici del vector), dopodiché viene eliminato tramite la funzione remove();. Viene utilizzata anche la funzione renameFiles per "shiftare" di 1 i numeri <n> dei nomi dei salvataggi successivi a quello eliminato.